

QuantFin

Case Study & Architecture / Workflow Document

Generated: 2026-04-26 15:49 UTC · Repository: abhi2k4/QuantFin

Summary. QuantFin is a full-stack quantitative finance decision-support platform targeting NSE-listed Nifty 50 equities. Built as both a research prototype and a working web application at A.P. Shah Institute of Technology, Thane, it unifies a workflow that is typically fragmented: OHLCV CSV ingestion → feature engineering (~70 technical indicators) → multi-model ML training (Ridge, Logistic, SVM, ARIMA, LSTM) → portfolio allocation → leakage-aware walk-forward backtesting → interactive React/TypeScript visualization. The backend is a FastAPI/Python service; the frontend is a Vite + React SPA with Clerk authentication and Recharts visualizations.

1. Repository Purpose, Domain Context & Primary User Journeys

1.1 Purpose

QuantFin is a **full-stack quantitative finance decision-support platform** targeting Indian large-cap equities (Nifty 50 constituents). It was built as a research prototype at **A.P. Shah Institute of Technology, Thane** (paper authors: Dr. Reshma Koli, Avanish Vadke, Atharva Shelke, Abhishek Thormothe, Aadit Suryawanshi, Prof. Harshada Sonkamble). The project is both a working web application and the implementation artifact behind an academic paper (*paper/quantfin_detailed.tex*, *paper/author.tex*).

The system unifies a workflow typically fragmented across isolated scripts: **CSV data** → **feature engineering** → **ML training** → **signal generation** → **backtesting** → **portfolio allocation** → **interactive web visualization**.

1.2 Domain Context

The platform focuses on **NSE-listed Nifty 50 stocks** (49 per-symbol CSV files in backend/data/). Data spans ~1991–2025 (~4,934 rows per symbol). CSVs use Yahoo Finance's 3-row multi-header export format.

1.3 Primary User Journeys

Journey	Entry Point	Key Interactions
Portfolio Manager	/dashboard	Loads portfolio summary (GET /api/portfolio/summary), rebalances via POST /api/portfolio/rebalance selecting LSTM/Linear/SVM/ARIMA and capital
Model Insights	/models	Polls GET /api/analytics/models, triggers background training via POST /api/analytics/models/train, polls training progress
Backtesting	/backtest	Submits POST /api/backtest/run with strategy, date range, capital, top-N, rebalance frequency; receives equity curve + metrics vs benchmark
Offline Research	QuantFin.ipynb	Jupyter notebook for data download (yfinance) and EDA; origin of the CSV dataset
Academic Paper	paper/*.tex	LaTeX source compiled with Springer svproc.cls; figures generated by paper/generate_backtest_figures.py calling the live backend

2. Repository Structure

Top-level layout of QuantFin/:

```
backend/ ← Python FastAPI service (all ML + data logic)
main.py ← FastAPI app factory, router registration, startup events
run.py ← CLI launcher (uvicorn wrapper)
requirements.txt ← Python dependencies (pinned)
data/ ← 49 per-symbol OHLCV CSVs (Nifty50, ~4934 rows each)
app/
routers/ ← Thin HTTP route handlers
analytics.py ← /api/analytics
backtest.py ← /api/backtest
portfolio.py ← /api/portfolio
predictions.py ← /api/predictions
services/ ← Core business logic
data_preprocessor.py ← CSV load, header normalisation, in-mem cache
feature_engineer.py ← ~70 technical indicators
ml_training_service.py ← Background training coordinator (async)
predict_service.py ← Ensemble prediction orchestrator
backtester.py ← Historical portfolio simulation engine
portfolio_allocator.py ← ML-driven allocation (prediction → weights)
portfolio_manager.py ← Live portfolio CRUD (SQLite-backed)
real_data_service.py ← CSV loader used by portfolio services
training_state.py ← JSON-backed training progress tracker
ml_models/ ← Per-model class implementations
linear_reg.py ← Ridge Regression (sklearn)
logreg.py ← Logistic Regression (sklearn)
svm_model.py ← SVM with RBF kernel (sklearn)
arima_model.py ← ARIMA/SARIMA (statsmodels)
lstm_model.py ← 2-layer LSTM (TensorFlow/Keras)
database/
portfolio_db.py ← SQLite schema + CRUD for portfolio persistence
models/schemas.py ← Pydantic request/response models
utils/
backtest_metrics.py ← CAGR, Sharpe, Sortino, MaxDD, Calmar, WinRate
scripts/ ← Offline training + inspection utilities
tests/ ← unittest suites (models, features, preprocessing)
training_summary.json / validation_report.json

src/ ← React/TypeScript frontend (Vite)
main.tsx ← App bootstrap (Router, ThemeProvider, Clerk)
App.tsx ← Route definitions + navigation shell
pages/ ← Dashboard, ModelInsights, BacktestPage, Landing ...
components/ui/ ← shadcn/ui component library
services/api.ts ← Centralised Axios client + typed API calls

paper/ ← Academic paper (Springer svproc, LaTeX + compiled PDFs)
quantfin_detailed.tex / author.tex / author2.tex
figures/ ← Generated backtest figures + summary JSON

QuantFin.ipynb ← Jupyter notebook (data download, EDA)
package.json ← Node.js dependencies + npm scripts
vite.config.ts ← Vite build config
README.md / context.md
```

3. Architecture Patterns

3.1 Python Backend — Layered Service-Oriented Architecture

The backend follows a strict layered pattern:

```
FastAPI Routers (HTTP boundary – no ML logic)
↓
Services (business logic, stateful singletons)
↓
ML Model Classes (train / predict / save / load)
↓
DataPreprocessor / RealDataService (data access)
↓
CSV Files / SQLite (persistence)
```

Services are singletons via module-level factory functions (e.g., `get_portfolio_allocator()` at `portfolio_allocator.py:475–480`). Each ML model class implements a uniform interface: `train()`, `predict()`, `save_model()`, `load_model()`. Artifacts are saved as **pickle** (sklearn/ARIMA) or **.h5 + pickle** (LSTM).

DataPreprocessor (`data_preprocessor.py`) provides in-process LRU caching via a `_cache` dict, handles the 3-row multi-header CSV format, and normalises OHLCV columns with forward/backward NaN fill.

3.2 TypeScript Frontend — SPA with Client-Side Routing

React 18 SPA built with Vite 5. State is local to each page component (`useState`). No global state manager — `@tanstack/react-query` is installed but not currently used. All API calls go through the typed `src/services/api.ts` Axios client. Clerk (`@clerk/clerk-react`) handles auth; UI primitives from `shadcn/ui` (Radix + Tailwind); charts from `Recharts`.

3.3 Notebook

`QuantFin.ipynb` is a Google Colab-originated notebook used to initially download Nifty 50 OHLCV data via `yfinance`. The saved state shows a `ModuleNotFoundError` in cell 1, meaning it was not run to completion. It is the **data acquisition origin** for the CSV files.

3.4 Academic Paper (LaTeX)

`paper/quantfin_detailed.tex` uses Springer Proceedings format (`svproc.cls`). It is reproducibility-anchored: all numeric claims are traced to saved JSON artifacts (`training_summary.json`, `paper/figures/*.json`). `paper/generate_backtest_figures.py` calls the live backend to produce figures.

4. End-to-End Workflows

4.1 Data Ingestion

```
[ORIGIN] QuantFin.ipynb → yfinance.download(Nifty50, period='10y')
↓
Saves per-symbol CSV (Yahoo Finance 3-row header)
↓
[STORAGE] backend/data/{SYMBOL}.csv (49 files, ~4,934 rows, 1991–2025)
```

CSV row structure: Row 0 = column names (Price, Close, High, Low, Open, Volume), Row 1 = ticker metadata, Row 2 = blank, Row 3+ = OHLCV data. `DataPreprocessor._handle_multi_row_header()` renames Price → Date and filters non-data rows.

4.2 Preprocessing (data_preprocessor.py:223–272)

Pipeline: cache check → CSV load → column validation → date parse → sort ascending → set Date index → forward/backward NaN fill → duplicate removal → numeric coercion.

4.3 Feature Engineering (feature_engineer.py:302–393)

~70 features stacked onto the base OHLCV DataFrame:

Category	Features Generated
Returns	returns (pct_change), log_returns
Rolling stats	rolling_mean/std/min/max/price_position × windows [7,14,21,30]
Momentum	momentum_N, roc_N × periods [5,10,20]
Moving averages	sma_N, ema_N, dist_from_sma_N × windows [5,10,20,50,200]
RSI	rsi (14-period), rsi_overbought (>70), rsi_oversold (<30)
MACD	macd, macd_signal, macd_histogram, macd_bullish, macd_bearish
Bollinger Bands	bb_middle/upper/lower, bb_width, bb_width_normalized, bb_position
Volume	volume_sma_5/20, volume_ratio, volume_momentum, obv
Price structure	high_low_range/pct, close_position, gap/gap_pct, atr_14

4.4 Model / Strategy Logic

Five model families, each a class in `backend/app/ml_models/`:

Class	Algorithm	Task	Key Design	Reported Metrics
LinearRegModel	Ridge Regression	21-day future return (regression)	72 indicators + StandardScaler + GridSearchCV α	test_r ² = −0.335 (RELIANCE)

Class	Algorithm	Task	Key Design	Reported Metrics
LogisticRegModel	Logistic Regression	Direction (up/down binary)	Same 72 features	test_accuracy = 50.0% (near-random)
SVMModel	SVM (RBF kernel)	Direction binary	GridSearchCV C, γ	test_accuracy = 51.4%, very high recall
ARIMAModel	ARIMA/SARIMA	Log-price time series	AIC/BIC grid ($p \in [0,5], d \in [0,2], q \in [0,5]$)	ADF stationarity check; 95% CI forecast
LSTMModel	2-layer LSTM	60-step sequential prediction	MinMaxScaler; LSTM(128)+Drop(0.2)+LSTM(64)+Drop(0.2)+Dense(1)	test_MAPE = 20.4%, test_r ² = -0.375 (RELIANCE)

4.5 Prediction / Allocation (portfolio_allocator.py)

`_get_raw_prediction()` implements **analytical prediction proxies** for each strategy (not loaded per-symbol pickle models):

- **LINEAR**: `scipy.stats.linregress` extrapolation on price history.
- **LSTM**: trend-following blend of `ret_5`, `ret_20`, `ret_60` + MACD histogram, scaled by volatility penalty.
- **SVM**: logistic sigmoid of momentum/RSI signal to derive P(up), mapped to expected return.
- **ARIMA**: mean-reversion signal relative to SMA20, scaled by RSI regime (`rsi_boost`).

Returns capped at [-30%, +50%]. Confidence computed from volatility, trend R², data recency, completeness.

Ranking and allocation: `score = predicted_return × confidence` → filter positive → top-N → weight proportional to score → integer share quantities.

4.6 Backtesting / Simulation (backtest.py:332–414)

Walk-forward loop with configurable monthly/quarterly rebalancing:

```
POST /api/backtest/run
→ generate_rebalance_dates(start, end, frequency)
→ for each period [t, t+1]:
  _get_all_predictions(strategy, forecast_days=30, current_date=t)
  ↑ data cutoff enforced (no leakage)
  select top-N by predicted_return (equal weight)
  calculate_period_return(stocks, weights, t, t+1) ← reads CSVs
  update capital
→ calculate_benchmark() (equal-weight Nifty50, same periods)
→ calculate_all_metrics() → CAGR, Sharpe, Sortino, MaxDD, Calmar, WinRate
```

Reported result (saved in `paper/figures/backtest_summary_2023_2024_linear.json`): LINEAR strategy, 2023-01-01 → 2024-01-01, monthly rebalancing: **CAGR 48.3%**, Sharpe 3.04, MaxDD -1.17% vs Benchmark **CAGR 30.6%**, Sharpe 1.75, MaxDD -4.18%.

■ *Interpretive caveat: metrics use analytical proxies (not loaded trained models) and simplified covariance/risk assumptions. The paper acknowledges the system is 'not production-ready' (validation_report.json).*

4.7 Visualization / Reporting

Frontend (Recharts in Dashboard.tsx, BacktestPage.tsx): portfolio equity AreaChart, strategy vs benchmark LineChart, position allocation PieChart. Academic figures generated by `paper/generate_backtest_figures.py` via matplotlib.

5. Runtime Flow & Component Interactions

5.1 Backend Startup Sequence

```
python run.py
→ uvicorn starts main.py:app
→ CORS middleware (localhost:3000/5173/5174)
→ request-timing middleware
→ startup_event():
DataPreprocessor() scans backend/data/*.csv → logs symbol count
initialize_prediction_services():
DataPreprocessor() → FeatureEngineer(preprocessor)
→ PredictionService(feature_engineer)
→ HTTP server ready on :8000
```

5.2 Key Component Dependency Graph

```
[HTTP Request]
↓
[Router: backtest / portfolio / analytics / predictions]
↓
[PortfolioAllocator]██████[RealDataService]██████[backend/data/*.csv]
█ █████[MLTrainingService]██████[models/*/model_latest.pkl]
█ █████[PortfolioDB]██████████████[backend/data/portfolio.db]
↓
[PredictionService]██████[FeatureEngineer]██[DataPreprocessor]██[*.csv]
█ █████[LinearRegModel]
█ █████[LogisticRegModel]
█ █████[SVModel]
█ █████[LSTMModel]
↓
[Backtester]██[DataPreprocessor + LinearRegModel]
↓
[backtest_metrics.py: calculate_all_metrics()]
```

5.3 Key Entry Points

Entry Point	File:Line	Role
app = FastAPI(...)	backend/main.py:30	App factory
startup_event()	backend/main.py:129	Warm-up: loads all symbols, initialises prediction services
DataPreprocessor.__init__	data_preprocessor.py:45	Sets data_dir to backend/data/, initialises cache
FeatureEngineer.generate_features()	feature_engineer.py:302	Master feature computation for one symbol
PortfolioAllocator.generate_allocations()	portfolio_allocator.py:29	Async: prediction → ranking → weighting → DB save
run_backtest()	backtest.py:332	FastAPI endpoint: orchestrates walk-forward simulation

Entry Point	File:Line	Role
simulate_strategy()	backtest.py:266	Core backtest loop (async, per-period, date cutoff enforced)
calculate_all_metrics()	backtest_metrics.py:252	7 risk/return metrics computed together
MLTrainingService.train_all_models()	ml_training_service.py	Background async training for all symbols

5.4 Data Persistence

Store	Technology	Path	Contents
Market data	CSV files	backend/data/*.csv	49 OHLCV files, 3-row header, 1991–2025
Trained models	Pickle	models/{type}/{symbol}/model_latest.pkl	sklearn/statsmodels model + scaler + metrics
LSTM weights	Keras H5	models/lstm/{symbol}/model_latest.h5	TF/Keras model weights
Portfolio state	SQLite	backend/data/portfolio.db	Positions, history, rebalance log
Training state	JSON	(via training_state.py)	Progress, model list, errors
Training artifacts	JSON	backend/training_summary.json	Last training run metrics (1 symbol committed)
Backtest figures	CSV/JSON/PNG	paper/figures/	Academic paper supporting data

6. Configuration, Environments & Build/Run Tooling

6.1 Backend Configuration (backend/.env.example)

API_HOST / API_PORT	0.0.0.0 / 8000
API_RELOAD	True (hot-reload via uvicorn)
ALLOWED_ORIGINS	http://localhost:3000, 5173 (CORS)
DATA_DIR	./data (CSV files)
MODEL_DIR	./models (trained artifacts)
LOG_LEVEL	INFO
ENABLE_CACHE	True / CACHE_TTL=3600
DATABASE_URL	Not configured — SQLite hardcoded to backend/data/portfolio.db

6.2 Frontend Configuration (.env.example, root)

VITE_API_BASE_URL	http://localhost:8000/api
VITE_USE MOCK_DATA	false
VITE_CLERK_PUBLISHABLE_KEY	Required — missing key causes startup crash (src/main.tsx)

6.3 Build and Run

```
# Backend
cd backend && pip install -r requirements.txt
python run.py # → http://localhost:8000
# Swagger UI: /docs ReDoc: /redoc

# Frontend
npm install && npm run dev # → http://localhost:5173 (Vite dev server)
npm run build # Production build → dist/
npm run lint # ESLint flat config

# Shortcut (from repo root)
npm run backend # cd backend && python -m uvicorn app.main:app --reload
```

Note: package.json script uses 'app.main:app' but run.py uses 'main:app'. Always run from the backend/ directory to avoid import errors.

6.4 Key Dependency Versions

Package	Version	Purpose
FastAPI	0.104.1	Backend web framework

Package	Version	Purpose
Uvicorn	0.24.0	ASGI server
Pandas	2.2.2	Data manipulation
NumPy	1.26.4	Numerical computation
Scikit-learn	1.4.2	Ridge, LogReg, SVM
TensorFlow	2.15.0	LSTM training
Statsmodels	0.14.1	ARIMA
XGBoost	2.0.3	Installed (README mentions it); not prominent in active code
yfinance	0.2.28	Data download (notebook only)
React	18.3.1	Frontend framework
Vite	5.4.2	Frontend build tool
TailwindCSS	3.4.10	Utility CSS
Recharts	3.2.1	Charts (line, area, pie)
Clerk	5.53.4	Authentication
Framer Motion	12.23.22	Page transitions

7. Testing / Evaluation & Reproducibility

7.1 Test Suites

File	Type	Scope
backend/tests/test_ml_models.py	unittest.TestCase	All 5 ML model classes with 500-row synthetic data
backend/tests/test_features.py	Demo/integration	Feature engineering via legacy services/features.py
backend/tests/test_preprocessing.py	unittest	DataPreprocessor loading + validation
backend/test_arima_accuracy.py	Ad hoc script	ARIMA accuracy evaluation
backend/test_rebalance.py	Ad hoc script	Rebalance logic test
backend/test_services.py	Ad hoc script	Service integration test
test_models_quick.py (root)	Ad hoc script	Quick model smoke test

7.2 Automated Validation Results (backend/validation_report.json)

Test	Result	Detail
Environment Setup	■ PASSED	49 data files found
API Route Registration	■ FAILED	Relative import error when run outside package context
Backtester Module	■ FAILED	0 passed, 0 failed (no test ran)
Integration Tests	■ FAILED	Backtester.__init__() missing positional arg 'models_dir'

Overall: 25% pass rate. production_ready: false (as of 2025-10-09).

7.3 Reproducibility

Training results for RELIANCE (4 models) are serialised in backend/training_summary.json and backend/training_results.json.

Backtest results stored in paper/figures/backtest_summary_2023_2024_linear.json and used directly in the LaTeX paper. paper/collect_paper_stats.py reads from these artifacts.

Limitation: Only RELIANCE appears in committed training artifacts. Training for all 49 symbols requires running backend/scripts/train_all_clean.py (~80 s/symbol × 5 models × 49 symbols ≈ 5.5 hours on CPU).

8. Deployment / Publishing Workflow

8.1 Web App (Current State)

There is **no containerisation, CI/CD, or cloud deployment configuration**. The system is designed for local development:

- Backend: `python run.py` or `uvicorn`
- Frontend: `npm run dev` (Vite dev) or `npm run build` + static hosting

8.2 Paper Publishing Workflow

```
paper/generate_backtest_figures.py
→ calls live backend /api/backtest/run
→ saves PNG figures + JSON summaries to paper/figures/

paper/collect_paper_stats.py
→ reads JSON artifacts → outputs stats for LaTeX tables

paper/build.ps1 (Windows PowerShell)
→ pdflatex / bibtex compilation
→ produces paper/quantfin_detailed.pdf, author.pdf, author2.pdf

Compiled PDFs committed to repo (viewable without LaTeX install)
```

9. Limitations, Technical Debt & Improvements

9.1 Structural / Architecture Issues

Issue	Location	Detail
Duplicate service classes	backend/app/services/ vs backend/services/	real_data_service.py and portfolio_db.py exist in both locations. backend/services/ are legacy stubs.
Parallel data service classes	data_preprocessor.py vs real_data_service.py	Both load the same CSVs with slightly different skip-row logic. Should be consolidated.
Hardcoded model base path	predict_service.py:48, linear_reg.py:41	Default models_dir='models/linear_reg' is relative; fails depending on CWD.
Import path issues	validation_report.json	'attempted relative import beyond top-level package' when scripts run outside package context.
Singleton without thread safety	portfolio_allocator.py:473–480	Global _portfolio_allocator set without lock; could race under concurrent requests.
ARIMA not wired into PredictionService	predict_service.py:25–26	# from app.ml_models.arima_model import ARIMAModel # TODO — commented out.
Backtester constructor mismatch	backtester.py:41	Requires models_dir as first positional arg; integration test doesn't supply it.

9.2 ML / Modelling Issues

Issue	Detail
Near-random classification	LogReg test accuracy 50.03%, SVM 51.36% (RELIANCE). Barely beat a coin flip — useful research baselines but not actionable signals.
Analytical proxies, not loaded models	PortfolioAllocator._get_raw_prediction() uses hand-crafted formulas, not serialised per-symbol models. The connection between train_all_clean.py output and allocation path is broken.
No covariance estimation	Portfolio risk = mean individual volatility. True portfolio risk requires a covariance matrix.
Regime blindness	No market regime detection (bull/bear). Single model applied regardless of regime.
Training only for RELIANCE	training_summary.json shows only 1 symbol trained; 48 symbols have no committed artifacts.
LSTM negative R ²	test_r ² = -0.375 means LSTM performs worse than predicting the mean. Needs more epochs/tuning.

9.3 Security / Data Issues

Issue	Detail
Auth token from localStorage	src/services/api.ts:12–15 — localStorage.getItem('auth_token') placeholder (TODO). API doesn't validate tokens server-side.
Path traversal risk	Symbol names used directly in file-path construction (data_dir / f"{symbol}.csv") — no sanitisation for user-supplied symbols.
SQLite in production	portfolio_db.py:17 — SQLite with check_same_thread=False; not suitable for multi-user production.

9.4 Concrete Improvement Opportunities

Improvement	Detail
Consolidate data loading	Merge DataPreprocessor + RealDataService; standardise 3-row header skip.
Wire trained models to allocation	In <code>_get_raw_prediction()</code> , load per-symbol pickle artifacts instead of analytical proxies.
Expand training to all 49 symbols	Run <code>backend/scripts/train_all_clean.py</code> and commit metrics JSONs.
Fix import paths	Add <code>sys.path</code> guards consistent with <code>tests/test_ml_models.py:21</code> ; run scripts from <code>backend/</code> CWD.
Covariance-based risk	Replace mean-volatility proxy with empirical covariance matrix (all close prices available).
Real-time data integration	Wire <code>RealDataService</code> to optionally fetch live prices via <code>yfinance</code> when CSV is stale.
Container deployment	Add <code>Dockerfile</code> for backend, <code>Nginx</code> static for frontend, <code>docker-compose.yml</code> .
Expand test coverage	Add integration tests for <code>router</code> → <code>service</code> → <code>data pipeline</code> (beyond synthetic-data unit tests).
Use TanStack Query	<code>@tanstack/react-query</code> is installed but unused — migrate <code>Dashboard/ModelInsights</code> fetch logic.

10. Architecture Diagram & Key File Reference

10.1 System Architecture (Text Diagram)



10.2 Key Files by Function

Function	Key File(s)
App entry (backend)	backend/main.py, backend/run.py
App entry (frontend)	src/main.tsx, src/App.tsx

Function	Key File(s)
Data loading	backend/app/services/data_preprocessor.py, real_data_service.py
Feature engineering	backend/app/services/feature_engineer.py
ML model classes	backend/app/ml_models/linear_reg.py, logreg.py, svm_model.py, arima_model.py, lstm_model.py
Model training (service)	backend/app/services/ml_training_service.py
Prediction orchestration	backend/app/services/predict_service.py
Allocation logic	backend/app/services/portfolio_allocator.py
Backtest engine	backend/app/routers/backtest.py, app/services/backtester.py
Backtest metrics	backend/app/utils/backtest_metrics.py
Portfolio persistence	backend/app/database/portfolio_db.py
Portfolio management	backend/app/services/portfolio_manager.py
API client (frontend)	src/services/api.ts
Dashboard UI	src/pages/Dashboard.tsx
Backtest UI	src/pages/BacktestPage.tsx
Model insights UI	src/pages/ModelInsights.tsx
Academic paper	paper/quantfin_detailed.tex (compiled → quantfin_detailed.pdf)
Saved training results	backend/training_summary.json, backend/training_results.json
Saved backtest results	paper/figures/backtest_summary_2023_2024_linear.json
Python dependencies	backend/requirements.txt
Node.js dependencies	package.json
Env config	.env.example (frontend), backend/.env.example (backend)
Tests	backend/tests/test_ml_models.py, test_features.py, test_preprocessing.py
Validation report	backend/validation_report.json
Context/hand-off doc	context.md